

Advanced System Architecture & Features Report

A comprehensive blueprint mapping the technology, folder organization, user/admin workflows, security remediations, and code quality systems of the Gemini-LMS platform.

PREPARED FOR

LMS Engineering Board

CREATED BY

Antigravity AI Engineer

DATE GENERATED

June 20, 2026

1. Executive Summary & Core Purpose

Gemini LMS is an advanced, AI-driven Learning Management System designed to orchestrate automated study material generation, student evaluation, and administrative moderation in real-time. By leveraging Google Gemini Pro and Flash models, the platform translates raw learning topics into complete, structured courses containing lecture summaries, chapters notes, flashcards, interactive quizzes, timed mock exams, and homework assignments.

The platform operates with a hybrid user interface: a Clerk-authenticated client dashboard for students/course creators, and a custom database-authenticated administrative panel for instructors, curators, and administrators. Background generation tasks are run via Inngest serverless event listeners, ensuring that heavy AI workloads do not block standard Next.js request threads.

CORE ARCHITECTURE GOALS

Ensure rapid outline rendering, offload notes compilation to cron workers, secure student routes against data leakage (BOLA/IDOR), and provide robust database backup layers without requiring bloated third-party cloud configurations.

2. System Technology Stack

Gemini LMS is built on a modern, robust Web stack optimized for serverless deployment and rapid horizontal scaling:

Layer	Technology Selected	Architectural Purpose
Framework	Next.js 15 (App Router, Node.js)	Supports high-performance hybrid rendering (SSR/SSG), API endpoints, route grouping, and streaming.
Database	Neon Serverless (PostgreSQL)	HTTP-based serverless Postgres queries, scale-to-zero capabilities, and connection caching.
ORM	Drizzle ORM	Type-safe database mapping, query builder syntax, schema declarations, and migration scripts.
Worker Jobs	Inngest	Event-driven serverless background workers, concurrency queues, step-retry handling, and scheduled cron jobs.
AI Integration	Google Generative AI (Gemini Pro/Flash)	Drives outline structuring, notes summarization, question generation, and homework grading.
Security / Auth	Clerk Auth & Custom Session Cookies	Dual authentication: Clerk for students; custom hashed session cookies for administrators.
Styling / UI	Tailwind CSS & Vanilla CSS	Responsive utility layouts paired with specialized custom CSS styles for glassmorphism and animations.
Notifications	Resend API & Sonner Toast	Triggers student reminders and grading updates; alerts client UI of real-time state changes.

3. Folder & File Organization

The codebase adheres to a strict, modular Next.js App Router workspace organization:

```

├─ app/                                # NEXT.JS ROUTE TREE
│   ├── (auth)/                        # Student login/signup routes via Clerk
│   ├── admin/                         # DB-authenticated Admin Portal routes
│   │   ├── dashboard/                # Admin Dashboard homepage
│   │   ├── backup/                  # Backup management interface [NEW]
│   │   └─ layout.jsx                 # Custom admin sidebar & session provider
│   ├── api/                          # BACKEND REST API ENDPOINTS
│   │   ├── (admin)/admin/            # Administrative APIs (auth checked)
│   │   │   ├── backup/              # Backup generate, history, and download [NEW]
│   │   │   └─ announcements/        # Announcements moderation
│   │   └─ (system)/inngest/          # Inngest integration worker route
│   ├── dashboard/                    # Clerk-authenticated Student Portal
│   │   ├── admin/                   # Clerk-auth Admin View
│   │   │   └─ backup/               # Backup page using shared component [NEW]
│   │   └─ explore/                  # Course exploration feed
│   ├── layout.js                     # Root HTML and fonts provider
│   └─ page.js                        # System landing page
├─ components/                        # REUSABLE FRONTEND COMPONENTS
│   ├── admin/                       # Admin panel dashboard cards and elements
│   │   ├── AdminPageShell.jsx        # Layout wrappers for admin pages
│   │   └─ BackupDashboard.jsx        # Glassmorphic database backup engine [NEW]
│   └─ ui/                           # Shadcn/Radix atomic UI components
├─ configs/                          # SYSTEM CONFIGURATION FILES
│   ├── db.js                        # Neon database connection driver
│   ├── schema.js                    # Drizzle Postgres schema definitions
│   └─ AiModel.js                    # Gemini AI models API connector
├─ hooks/                            # CLIENT-SIDE CUSTOM REACT HOOKS
├─ inngest/                          # BACKGROUND ACTION WORKERS
│   ├── client.js                    # Inngest event registration configuration
│   ├── functions/                   # Modular background job controllers
│   │   ├── cronJobs.js              # System health, course cleaner, weekly cron
│   │   └─ course.js                 # Notes generation, youtube mapping worker
│   └─ functions.js                  # Aggregate exports wrapper
├─ lib/                              # CORE HELPER UTILITIES
│   ├── adminAuth.js                 # Hashing (PBKDF2) and session encryption
│   └─ adminApiAuth.js               # API authorization middleware checks
└─ public/                           # STATIC ASSETS (logos, icons, certificates)

```

4. Detailed Features: User/Student Section

The student portal revolves around learning, quiz evaluation, peer collaboration, and tracking progress:

4.1. Course Outline & Content Generation

Students input a desired topic and course layout preference. The system calls the Gemini API to construct a structured course outline containing chapters, study modules, and descriptions. This outline is reviewable and can be generated into sub-contents asynchronously.

4.2. Chapter Notes Summary

Once the outline is approved, an Inngest background event triggers the notes generation worker. The worker queries Gemini to compile highly detailed markdown pages for each chapter, caching them in the `chapterNotes` table.

4.3. Interactive Quizzes & Summaries

The student can request interactive study contents. For each chapter, the system generates study materials:

- **Flashcards:** Fully interactive double-sided cards rendering study terms with 3D flip animations.
- **Practice Quizzes:** Support for Multiple Choice, True/False, Short Answer, and Fill-in-the-blank questions dynamically evaluated with scores.

4.4. Timed Mock Exams

Provides complete, timed evaluations mimicking actual exams. The exam runs with a countdown clock and grades answers on submit, saving scores in the `mockExamSubmissions` table.

4.5. Chapter Peer Discussions

Every course chapter includes a collaborative forum. Students write questions or feedback, and other learners or course tutors reply in real-time, linked by discussion and reply foreign keys.

4.6. Assignment Editor & Submission

Students can submit homework drafts as rich text, code (with language labels), or document links. The editor includes a real-time markdown split preview pane.

4.7. Adaptive Difficulty Adjustment

Tracks performance scores per student/topic. If a student falls below 60% after multiple attempts, the topic is flagged, and the quiz engine automatically downshifts the difficulty level (e.g. from Hard to Medium) to reinforce basic concepts, adjusting up as the student improves.

4.8. Parent Portal Tokens

Allows parents to monitor child performance. Students generate a secure parental access token. Parents use this token to access a clean view containing grade summaries, active assignments, and tutor remarks without editing capabilities.

5. Detailed Features: Administrative Section

The administrative workflows secure, moderate, audit, and safeguard the LMS:

5.1. Operations Command Dashboard & Analytics

Renders live metrics: total registered users, credits spent, generations running, failed runs, monthly revenue logs, and support queue backlogs.

5.2. Course Content Moderation & Review Queue

Curators view course outlines. Admins can edit generated content, approve outline layouts, override AI notes, or flag inaccurate study questions before publishing.

5.3. Announcements Manager

Allows super-admins to deploy system-wide banner notifications. Targets can be filtered (all users, students only, tutors only) and pinned, supporting automatic expiration timestamps.

5.4. Submissions & Manual Regrading Queue

Displays all student assignment submissions. Tutors can read drafts, review the automated AI grade/feedback, and apply a manual override score with instructor remarks.

5.5. Team Access & Hashed Password Manager

Super-admins add team staff (admins/tutors). The system creates a secure activation token. When the staff member clicks the link, they set their credentials, which are cryptographically hashed using PBKDF2.

5.6. Tutor Applications Moderation

Normal users apply to become tutors. Administrators review the motivation letters, subject expertise, and qualifications queue to approve or reject requests.

5.7. Audit Logs & Activity Tracking

Logs every administrative action (grades manually overridden, credits adjusted, passwords reset, courses deleted) alongside IP address and timestamps for strict accountability.

5.8. Premium System Data Backup

Protects platform data by packing all tables into compressed, downloadable archives:

- **Manual Snapshots:** On-demand generation compiles 16 database tables, compresses them with Gzip, hashes a transient token, and logs the metadata.
- **Scheduled Backups:** An Inngest cron job automates this process weekly.
- **Direct Downloads:** Serves `.json.gz` attachments securely using verified transient download tokens.

6. Security Architecture & Audit Remediation

We recently executed a comprehensive security audit and remediation to secure all API endpoints against BOLA, IDOR, Stored XSS, and authorization lockouts:

FIXED

BOLA & Spoofing

Previously, client-supplied email arguments in payloads were trusted. Endpoints (``DELETE /api/course``, ``/api/user-streak``, ``/api/submit-assignment``) now query active Clerk session claims to guarantee students can only delete or read their own data.

FIXED

Admin Endpoint Guards

All administrative routes are wrapped with ``@/lib/adminApiAuth`` middleware. Tutors and admins are checked for valid session cookies (``admin_session``) to block unauthenticated access.

FIXED

Tutor Lockout Bug

Aligned password hashing in ``/api/admin/auth/set-password`` (previously using mismatching `bcryptjs`) to `PBKDF2Sync`. Temporary passwords are automatically cleared upon verification to prevent credentials leak.

FIXED

DOM / Stored XSS

Integrated ``DOMPurify.sanitize()`` inside the assignment split editor preview pane to neutralize scripts and sanitizes markdown inputs before rendering.

7. Code Quality & Development Standards

The codebase implements modular code guidelines to ensure maintainability:

- **Drizzle Schemas:** Centralized table definitions in ``configs/schema.js`` ensure type-safe database queries.
- **Session Helpers:** Admin checks are centralized in ``lib/adminApiAuth.js`` to prevent validation logic sprawl.
- **Separation of Concerns:** React pages utilize hooks (e.g. ``useAdminDashboard``) for state management, keeping layout code clean.
- **Gzip Compression:** Backup files utilize Node's built-in ``zlib`` to compress text files into standard ``.gz`` archives, saving bandwidth and space.

INTEGRATION TEST VERIFICATION

The backup system was verified using a standalone integration script that successfully queries all 16 database tables, compresses them into an archive, registers logs, and decodes/decompresses the file to confirm payload integrity.